

A Security Perspective on Retrieval Augmented Generation

Legal Domain Case Study: Complete Implementation + Hardening

Presented by: Jasveer Panwar

Guided By: Dr. B.L. Parne

Department of CSE, SVNIT Surat

Contents

- 1 Introduction
- 2 Problem Statement
- 3 Motivation
- 4 Literature Survey
- 5 Research Gap
- 6 Objectives
- 7 Architecture
- 8 Implementation
- 9 Implemented Security Features
- 10 Improvements Over Previous Stage
- 11 Current Findings
- 12 Limitations and Future Work
- 13 Conclusion
- 14 References

- Retrieval-Augmented Generation (RAG) increases factual grounding for legal QA.
- Security risk remains high if retrieved context is malicious or low-trust.
- This work builds a legal RAG first, then hardens it with defense-in-depth controls.

Problem Statement

- Plain LLMs are not enough for reliable legal answers.
- RAG helps, but trust in corpus and retrieval is a major weak point.
- Aim: maintain relevance while reducing poisoning and prompt-injection risk.

Motivation

- Legal updates are frequent (BNS/BNSS era changes).
- Hallucinated legal content is unacceptable.
- Query-time security and auditability are essential for high-stakes usage.

Literature Survey

Paper	Contribution	Limitation
Backdoored Retrievers	Shows retriever-level backdoor and injection risks	Strong on attacks, less operational defense detail
RAG Security Survey	Full threat taxonomy across RAG stages	More conceptual than implementation-level
PoisonedRAG	Demonstrates high poisoning success rates	Limited production hardening blueprint
Securing RAG Framework	Risk-mitigation mapping	Limited empirical defense validation

- Lack of end-to-end legal RAG hardening in runnable codebases.
- Weak integration of trust priors and provenance signals in reranking.
- Missing combined design: retrieval defense + prompt defense + logging.

Objectives

- Build complete legal RAG pipeline.
- Add practical security controls in live query path.
- Keep backend model-agnostic (OpenAI + Ollama).
- Provide observability and reproducible logs.

Missing image: rag.jpg

Implementation Modules

- `src/ingest_with_metadata.py`: parsing + metadata + chunking
- `src/embed_index.py`: embeddings + FAISS index
- `src/serve_query.py`: secure retrieval + rerank + prompt + model call
- `src/interactive_openai.py`: interactive inspection
- `src/evaluate.py`, `src/comapre_models.py`: evaluation and comparison

Reproducible Commands

- `python src/ingest_with_metadata.py --data_dir data --out_file chunks.jsonl --chunk_size 1200 --overlap 300`
- `python src/embed_index.py --chunks chunks.jsonl --model sentence-transformers/all-MiniLM-L6-v2 --index_dir index --batch 64`
- `python src/serve_query.py --question "When did CrPC come into effect?" --model openai:gpt-4o-mini --k 8 --min_sim 0.12 --strict`

Implemented Security Features

- Query normalization for legal shorthand expansion.
- Prompt-injection chunk filtering before context assembly.
- Trust-aware reranking with untrusted TXT demotion.
- Context-only answering in strict mode.
- Refusal output when evidence is missing: “I don’t know”.
- Query-wise JSON logging for diagnostics and audit.

Reranking Equation

$$S_{combined} = S_{embed} + \alpha S_{filename} + \beta S_{phrase} + \gamma S_{meta} + \delta S_{trusted} - \lambda S_{untrusted}$$

- Keeps semantic relevance from embeddings.
- Adds legal lexical precision and metadata alignment.
- Lowers prominence of potentially poisoned low-trust content.

Threat to Control Mapping

Threat	Implemented Control
Indirect prompt injection	Injection chunk filtering + anti-instruction prompt note
Corpus poisoning	Trust-aware reranking + untrusted TXT penalty
Query ambiguity	Query rewrite for legal abbreviations
Ungrounded output	Context-only policy + refusal mode
Low traceability	Structured logs with retrieval diagnostics

Improvements Over Previous Stage

- Earlier stage showed attacks and baseline behavior.
- Current stage deploys active defenses in production query flow.
- Added trust policy, strict prompting, and detailed diagnostics.
- Improved maintainability with modular scripts.

Current Findings

- Retrieval is mostly statute-aligned for domain queries.
- Injection-like chunks are reduced before prompt creation.
- Logs provide transparent evidence for each answer.
- Status: strong engineering baseline; adversarial benchmark next.

Limitations

- Heuristic filters can miss obfuscated attacks.
- Trust priors require governance and source policy quality.
- Formal ASR reduction benchmark is pending.

Future Work

- Automated poisoning/backdoor benchmark harness.
- Learned sanitizer and anomaly detection.
- Source signature/provenance verification.
- Refusal calibration and multilingual expansion.

Conclusion

- Complete legal RAG implemented and hardened.
- Security now spans retrieval, reranking, prompting, and logging.
- Work provides practical deployment-ready defense defaults.

References

- Backdoored Retrievers for Prompt Injection Attacks on RAG, arXiv 2024.
- PoisonedRAG: Knowledge Corruption Attacks to RAG, USENIX Security 2025.
- RAG Security Challenges and Countermeasures, IEEE IoT Journal 2025.
- Securing RAG: Risk Assessment and Mitigation Framework, IEEE TIFS 2024.
- Retrieval-Augmented Generation for Knowledge-Intensive NLP, NeurIPS 2020.

Thank You

Questions?